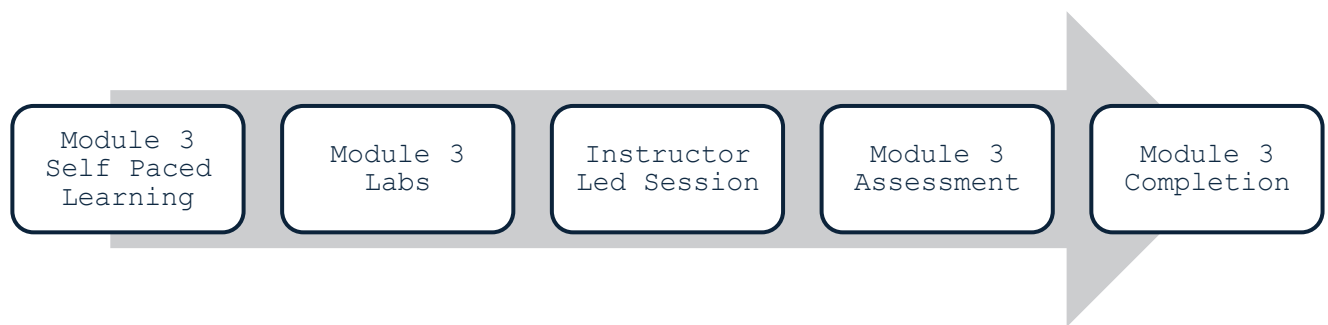


AI Pioneers: Claude Certified Architect – Foundations (CCA-F)

Module 3 — Claude Code Configuration & Workflows

Overall Plan



Self-Paced Learning Plan

Module 3 introduces no new Skilljar courses — it builds on courses you completed in Modules 1–2. The supporting courses (and where you completed them) are listed below; revisit the specific lessons in the section-by-section plan.

Supporting course	Skilljar link
Claude Code 101 — completed in Module 1	https://anthropic.skilljar.com/claude-code-101
Claude Code in Action — completed in Module 2	https://anthropic.skilljar.com/claude-code-in-action
Introduction to Agent Skills — completed in Module 1	https://anthropic.skilljar.com/introduction-to-agent-skills

Key Sections: Claude Code Configuration & Workflows

Sec	Section	Specific lesson(s) to watch
S1	CLAUDE.md Configuration Hierarchy & Modularity	• Claude Code 101: The CLAUDE.md file
S2	Custom Slash Commands & Skills	• Claude Code in Action: Custom commands • Claude Code 101: Skills • Introduction to Agent Skills: Creating your first skill; Configuration and multi-file skills; Sharing skills (supporting)
S3	Path-Specific Rules & Conditional Loading	• Claude Code 101: The CLAUDE.md file • Introduction to Agent Skills: Configuration and multi-file skills (supporting)
S4	Plan Mode vs Direct Execution	• Claude Code 101: The explore → plan → code → commit workflow • Claude Code in Action: Adding context; Controlling context (supporting)
S5	Iterative Refinement Techniques	• Claude Code in Action: Making changes • Claude Code 101: Code review
S6	CI/CD Integration with Claude Code	• Claude Code in Action: Github integration; The Claude Code SDK

Labs

Lab 3.1 — Configuring Claude Code: CLAUDE.md Hierarchy, Commands & Skills (mandatory)

- Layer CLAUDE.md across levels with @import and modular rules — e.g. a global style rule plus a repo-specific testing convention.
- Build custom slash commands for repeatable actions — e.g. a /review command that runs your team's standard checklist.
- Package reusable Skills so common workflows are one invocation away — e.g. a “generate API docs” skill shared across projects.

Lab 3.2 — Targeted Behavior: Path-Specific Rules & Plan Mode Workflows (mandatory)

- Load conditional rules by glob path so context stays relevant — e.g. applying strict security rules only under /auth/**.
- Use Plan mode to design before executing on risky changes — e.g. planning a multi-file migration before any edits land.
- Deploy the Explore subagent to map unfamiliar code first — e.g. surveying a new module before proposing changes.

Lab 3.3 — From Refinement to Pipeline: Iterative Workflows & CI/CD Integration (Optional)

- Refine via test-driven and interview-style loops — e.g. writing failing tests, then iterating until they pass.
- Run headless Claude Code in CI/CD for automated checks — e.g. a pipeline step that reviews every pull request.
- Emit structured JSON output for downstream automation — e.g. parsing review results into a pass/fail gate.

(Refer to the lab document over LMS for detailed information)

Instructor Led Session

Key Considerations before Instructor Led Session

Make sure you can answer these from your self-paced study:

- How does the CLAUDE.md hierarchy resolve across user / project / directory levels, and what do @import and /memory do?
- When do you use a slash command vs a Skill vs an always-on CLAUDE.md rule, and where does each live?
- How do path-specific rules (glob frontmatter in .claude/rules/) save tokens and improve precision?
- When do you use Plan mode vs direct execution, and what is the Explore subagent for? How do you run Claude Code headlessly in CI/CD (-p, --output-format json), and how do you review only new issues?

What the instructor-led session covers

Skilljar teaches the concepts; the 60-minute live lab is where you build and debug:

- Build a multi-level CLAUDE.md with @import and verify it with /memory (S1).
- Create a project slash command and a SKILL.md skill; configure context:fork and allowed-tools (S2).
- Add path-scoped rules in .claude/rules/ with glob frontmatter; compare vs a monolithic CLAUDE.md (S3).
- Use Plan mode for a large refactor and the Explore subagent for discovery; direct execution for a quick fix (S4).
- Apply test-driven and interview-style refinement loops (S5).
- Run Claude Code with -p in CI and emit JSON output for a pass/fail gate (S6).

Section-by-section study plan

S1 — CLAUDE.md Configuration Hierarchy & Modularity	
Key concepts	3-level config: user (~/.claude/CLAUDE.md), project (.claude/CLAUDE.md), and directory — lower (more specific) wins. Use @import to split config into modules; debug with /memory.
Watch on Skilljar (specific lessons)	• Claude Code 101 — The CLAUDE.md file Note: The 3-level hierarchy (user / project / directory), @import modularity, and /memory debugging are reinforced in the lab.
Hands-on activity (try after watching)	Create a multi-level CLAUDE.md; use @import; move rules to .claude/rules/; verify with /memory.
Suggested self-paced time	45 min

S2 — Custom Slash Commands & Skills	
Key concepts	Reusable commands live in .claude/commands/ (project, shared via Git) or ~/.claude/commands/ (personal). Skills in .claude/skills/SKILL.md run on-demand; CLAUDE.md rules are always-on.
Watch on Skilljar (specific lessons)	• Claude Code in Action — Custom commands • Claude Code 101 — Skills • Introduction to Agent Skills — Creating your first skill; Configuration and multi-file skills; Sharing skills (supporting) Note: Slash commands come from Claude Code in Action; Skills from Claude Code 101 and the Agent Skills course. context:fork and allowed-tools are configured in the lab.
Hands-on activity (try after watching)	Create a project slash command; build a SKILL.md skill; configure context:fork and allowed-tools; test skill vs default.
Suggested self-paced time	60 min

S3 — Path-Specific Rules & Conditional Loading

Key concepts	Define rules in <code>.claude/rules/</code> with YAML frontmatter + glob patterns (e.g., <code>**/*.test.tsx</code>). Rules load only when paths match — saving tokens and improving precision.
Watch on Skilljar (specific lessons)	• Claude Code 101 — The CLAUDE.md file • Introduction to Agent Skills — Configuration and multi-file skills (supporting) Note: Path-scoped rules in <code>.claude/rules/</code> with glob frontmatter have no dedicated video — they are built in the lab on top of the CLAUDE.md config model.
Hands-on activity (try after watching)	Create path-scoped rules; apply them to test and API files; validate conditional loading; compare vs a monolithic CLAUDE.md.
Suggested self-paced time	45 min

S4 — Plan Mode vs Direct Execution

Key concepts	Use Plan mode for complex multi-file tasks and direct execution for simple, clear changes. Hybrid: Plan → Execute. Use the Explore subagent to isolate the discovery phase.
Watch on Skilljar (specific lessons)	• Claude Code 101 — The explore → plan → code → commit workflow • Claude Code in Action — Adding context; Controlling context (supporting) Note: Plan mode and the Explore subagent are applied in the lab; the <code>explore→plan→code→commit</code> lesson establishes the plan-first model.
Hands-on activity (try after watching)	Use plan mode for a large refactor; direct execution for a bug fix; use the Explore subagent; compare output quality.
Suggested self-paced time	45 min

S5 — Iterative Refinement Techniques

Key concepts	Give concrete input/output examples. Write tests first (TDD), then refine code based on failures. Use interview-style prompting to clarify before execution.
Watch on Skilljar (specific lessons)	• Claude Code in Action — Making changes • Claude Code 101 — Code review Note: <i>TDD and interview-style prompting are practised in the lab; these lessons cover the iterate-on-changes and review loop.</i>
Hands-on activity (try after watching)	Provide I/O examples; implement a test-driven workflow; use interview-style prompting; fix multiple issues with different strategies.
Suggested self-paced time	45 min

S6 — CI/CD Integration with Claude Code

Key concepts	Run Claude Code non-interactively with the -p flag in pipelines. Get structured JSON with --output-format json. Use CLAUDE.md as CI context; review only new issues each run.
Watch on Skilljar (specific lessons)	• Claude Code in Action — Github integration; The Claude Code SDK Note: <i>The -p headless flag, --output-format json, and pipeline review are built in the lab; the SDK and GitHub integration lessons are the foundation.</i>
Hands-on activity (try after watching)	Run Claude Code with -p in CI; generate JSON output with a schema; integrate the review workflow into a pipeline.
Suggested self-paced time	45 min

